

Operating Systems Lab

Offline Assignment 4 — xv6 Scheduling

A Multilevel Feedback Queue with Lottery Scheduling and Aging

Overview

The default xv6 scheduler is a plain round-robin: it walks the process table and runs whichever process it finds first in the `RUNNABLE` state, giving every process the same share of the CPU regardless of how it behaves. In this assignment you will replace it with a two-level **Multilevel Feedback Queue (MLFQ)**¹ scheduler:

- **queue 0** (the top queue) schedules its processes using **lottery scheduling**².
- **queue 1** (the bottom queue) schedules its processes using plain **round-robin**.
- An **aging** mechanism promotes processes that have been waiting too long in **queue 1**, so that no process starves.

You will implement this scheduler almost entirely inside `kernel/proc.c`, add two new system calls (`settickets` and `getpinfo`), and write two user-space programs to exercise and observe it.

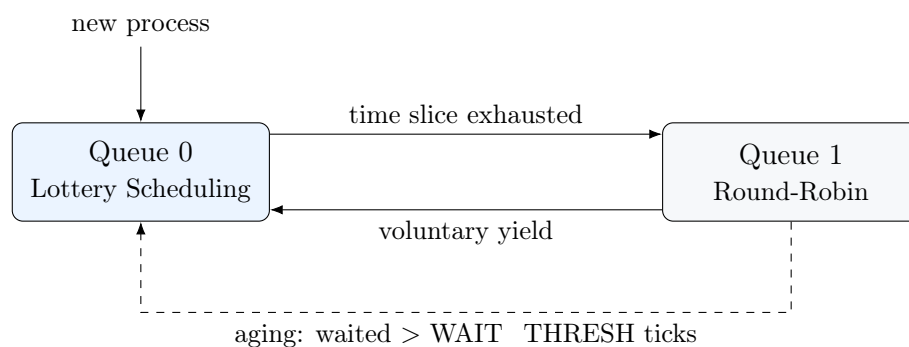
How the Scheduler Behaves

Conceptually, a process moves between the two queues according to a small set of rules:

1. A newly created process always starts in **queue 0**, the highest-priority queue.
2. To decide who runs next, the scheduler searches the queues top-down. The first non-empty queue it finds supplies the next process, chosen according to *that queue's own* algorithm (lottery for **queue 0**, round-robin for **queue 1**).
3. Once a process leaves the CPU — for any reason — the scheduler forgets where it left off and restarts its search from **queue 0** again.
4. What happens to a process when it leaves the CPU depends on *why* it left:
 - If it finishes its work inside its current queue's time limit, it simply exits, as usual.
 - If it uses up its *entire* time slice without finishing, it is preempted and pushed down to the next lower queue.
 - If it voluntarily gives up the CPU before its time slice is used up (e.g. it calls `sleep` to wait for I/O) it is promoted one level up. A process voluntarily yielding from **queue 1** moves to **queue 0**.
5. Independently of the above, any process that has been waiting in **queue 1** for longer than a threshold is promoted to **queue 0** by the **aging** mechanism (Section 5), so that low-priority processes cannot be starved forever.

¹<https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-mlfq.pdf>

²https://en.wikipedia.org/wiki/Lottery_scheduling



Timer Ticks vs. Time Slices

xv6 already keeps a global clock: the `ticks` variable in `kernel/trap.c`, incremented on every timer interrupt. Under the vanilla round-robin scheduler, each pass through the scheduler loop runs one process for exactly one tick before moving to the next; with processes A, B, and C, the resulting pattern is **A B C A B C ...**

For MLFQ, a queue's time slice is a number of *ticks*, not a single scheduler-loop iteration. **queue 0** and **queue 1** allow a process to run for several ticks in a row before it is forced to move down a queue.

This part is easy to get wrong

In xv6, a context switch happens on *every single* timer interrupt — a process cannot simply "hold the CPU" for multiple ticks in a row inside one call to the scheduler. Instead, the process repeatedly gives control back to `scheduler()` and the scheduler must recognize that this same process still has time slice left, and choose it again. Concretely: suppose A and B are both in **queue 0**, and **queue 0**'s time slice is 2 ticks. When A is scheduled, it does *not* run uninterrupted for 2 ticks — it yields back to the scheduler after every tick, exactly as in plain round-robin. What changes is the scheduler's decision: it must keep resuming A, tick after tick, until A has accumulated its full 2-tick allowance, and only then move on to B.

Make sure you track, per process, how many ticks it has consumed *within its current turn at its current queue*, and reset that counter whenever the process changes queue or is freshly scheduled after giving up the CPU.

Lottery Scheduling

Each process holds a fixed, integer number of *tickets*. Instead of taking turns deterministically, the scheduler holds a lottery: a process's probability of winning is proportional to its share of the total tickets currently held by all runnable processes at that instant. If processes $p_1, \dots, p_n$ currently hold $t_1, \dots, t_n$ tickets, the probability that p_i wins is

$$\Pr[p_i] = \frac{t_i}{\sum_{j=1}^n t_j}.$$

You do not need floating-point division to implement this. Follow this procedure instead:

1. Sum the current ticket counts of all runnable processes in **queue 0**; call this total n .

2. Draw a random integer r uniformly from $(0, n]$. This draw *must* be uniform over the interval — do not use any other distribution.
3. Walk the processes while keeping a running sum of their current ticket counts. The process whose ticket count first causes the running sum to exceed r is the winner.

Every time a process wins the lottery and is scheduled, decrement its current ticket count by one. Ticket counts are therefore consumed over time; once *every* runnable process's current ticket count has reached zero, reset all of them back to their original ticket counts and continue.

Aging

Aging protects against starvation: a process that has been waiting too long in a lower-priority queue is gradually promoted to a higher one. A process already sitting in the highest queue it can reach needs no adjustment.

Aging is not priority boosting

It is easy to conflate these two ideas, but they solve the same problem differently:

- **Aging** looks at individual processes and promotes only those that have personally waited too long.
- **Priority boosting** ignores individual wait times and periodically promotes *every* process back to the top queue, all at once.

This assignment asks for *aging*, not priority boosting.

Specification Parameters

Define all of the following as macros in `kernel/param.h`:

Macro	Value	Meaning
TIME_LIMIT_0	2	Time slice, in ticks, a process is allowed while in queue 0 .
TIME_LIMIT_1	4	Time slice, in ticks, a process is allowed while in queue 1 .
WAIT_THRESH	6	Ticks a process may wait before aging promotes it to queue 0 .
DEFAULT_TICKETS	10	Ticket count every process starts with.

A few simplifications apply for this assignment:

- Assume a **single CPU**. Set `CPUS := 1` in the xv6 Makefile.
- You do not need a rigorous enqueue/dequeue data structure for the two queues. It is enough to store a queue number on each process and, when searching for work, look at **queue 0** first and **queue 1** second.

You may add whatever fields you need to `struct proc` (in `kernel/proc.h`) to track this state — for example, the process's current queue, its original and current ticket counts, and how many ticks it has consumed in its current turn. Initialize these fields when a process is created, and update them whenever the process changes queue, is scheduled, or wins the lottery.

System Calls

You will add two new system calls.

settickets

```
int settickets(int number);
```

Sets the number of tickets held by the *calling* process. Every process starts with `DEFAULT_TICKETS` tickets by default; calling `settickets` lets a process request a different share of the CPU.

- Returns 0 on success.
- Returns -1 if `number` is less than 1. In that case, the process's ticket count is left at (or reset to) `DEFAULT_TICKETS`.

getpinfo

```
int getpinfo(struct pstat *);
```

Fills in a `struct pstat` with a snapshot of every active process: PID, status, current queue, and scheduling statistics. This is the syscall a user-space `ps`-like tool would call.

- Returns 0 on success.
- Returns -1 if given a bad or NULL pointer.

Add a new header, `kernel/pstat.h`, containing exactly the following structure — it must not be modified:

```
#ifndef _PSTAT_H_
#define _PSTAT_H_
#include "param.h"

struct pstat {
    int pid[NPROC];           // process ID of each process
    int inuse[NPROC];         // whether this proc slot is in
                             // use (1 or 0)
    int inq[NPROC];           // current queue of the process
    int waiting_time[NPROC];  // ticks spent waiting before
                             // being scheduled
    int running_time[NPROC];  // times scheduled before time
                             // slice was used
    int times_scheduled[NPROC]; // total times this process was
                             // scheduled
    int tickets_original[NPROC]; // tickets originally assigned
    int tickets_current[NPROC];  // tickets currently held
    uint queue_ticks[NPROC][2]; // total ticks spent in each queue
};

#endif // _PSTAT_H_
```

Files You Will Touch

File	What changes
kernel/proc.c	The scheduler itself: queue transitions, lottery draws, aging, and ticket inheritance in <code>fork()</code> .
kernel/proc.h	New per-process fields on <code>struct proc</code> (current queue, ticket counts, ticks consumed, ...).
kernel/param.h	The four macros from Section 4.
kernel/defs.h	Prototypes for any new functions you add. If a function takes a struct argument, declare that struct at the top of <code>defs.h</code> .
kernel/pstat.h	New file — the <code>struct pstat</code> definition above.
user/dummyproc.c	New test program (Section 9).
user/testprocinfo.c	New test program (Section 9).

Most of the scheduler logic is localized inside `proc.c`; start by reading and understanding the existing round-robin `scheduler()` function there before changing anything.

Implementation Notes

Ticket inheritance on fork

A child process must inherit its parent's *current* ticket count. If a parent has 17 tickets when it calls `fork()`, its child starts out with 17 tickets too. Set this up inside `fork()` in `kernel/proc.c`, where the child's `struct proc` is initialized.

Passing arguments between user space and the kernel

Follow the pattern used by existing system calls — tracing `read()` down into `sys_read()` shows how `argaddr()` (and related helpers) retrieve a pointer passed in from user space. Adding a new system call also means wiring it up the same way existing ones are wired (syscall number, prototype, user-space stub) — mirror what `read()` does throughout.

Pointers coming from user space are a security hazard: validate them carefully before dereferencing. To copy data back out to user space, use `copyout()`.

Random number generation

You need a pseudo-random number generator inside the kernel for the lottery draw. You may write your own or adapt a public-domain implementation. Two requirements: it must run entirely at kernel level, and it must use a *deterministic seed* so that a given run is reproducible.

Testing

Write two user-space programs to exercise your implementation.

`dummyproc`

Calling syntax:

```
$ dummyproc <tickets> <iterations>
```

For example, `dummyproc 43 100000` requests 43 tickets (via `settickets`) and runs a dummy loop for 100000 iterations.

`dummyproc` should also fork a child that runs a similar dummy loop, except the child periodically sleeps partway through its iterations — simulating a process that voluntarily gives up the CPU before its time slice is exhausted. This is what lets you trigger and observe a `PROMO` event (Section 10).

`uint32` in xv6 tops out at 4294967295. Make sure your loop counters cannot overflow past this value, or the loop will never terminate.

If `testprocinfo` does not see your process

If `testprocinfo` does not list a `dummyproc` you just launched, it most likely finished before you could observe it. Run it for at least a billion iterations, or nest an outer loop around the dummy work, to keep it alive long enough to inspect.

`testprocinfo`

Calling syntax:

```
$ testprocinfo
```

Takes no arguments. It calls `getpinfo`, then prints the resulting `struct pstat` in a readable, `ps`-like format.

Sample session

```
$ dummyproc 10 1000000000 &
$ dummyproc 5 1500000000 &
$ testprocinfo
```

`testprocinfo` should print output along these lines (illustrative values):

PID	In Use	In Q	Waiting time	Running time	Times sched.	Orig. tix	Curr. tix	q0 ticks	q1 ticks
1	0	0	0	1	34	10	9	344	0
2	0	0	0	0	30	9	8	343	0
41	1	0	0	1	1	8	7	0	0
21	1	0	8	1	12	5	4	91	34
23	1	1	3	0	20	10	8	63	61

Column meanings:

- **In Use** — whether the process is `RUNNING` (1) or `RUNNABLE` (0).
- **In Q** — the process's current queue.
- **Waiting time** — ticks spent before being scheduled; increment this only while the process is `RUNNABLE`.
- **Running time** — ticks spent before the process used up its allocated time slice.

- **Times scheduled** — total number of times the process was scheduled by the CPU.
- **q0 / q1 ticks** — total ticks the process has spent in queue 0 / queue 1, regardless of its current state.

Do not print processes whose PID is 0.

Logging

Keep a global toggle (e.g. `print_logs`) that turns the following diagnostic messages on or off. Because the scheduler may print these in the middle of ordinary program output, format them so they stand out — ANSI color codes can help here.

- **Demotion** — printed when a process uses up its time slice and moves to a lower queue:

```
DEMO: Process 1 (sh) ran for 2 time ticks, demoted to queue 1
```

- **Voluntary promotion** — printed when a process gives up the CPU early (e.g. waiting for I/O, or calling `sleep`) and moves to a higher queue:

```
PROMO: Process 1 (sh) ran for 3 time ticks, promoted to queue 0
```

This is what your `dummyproc` child (Section 9.1) should trigger by sleeping partway through its iterations.

- **Aging promotion** — printed when a process is promoted purely because it waited past `WAIT_THRESH`:

```
BOOST: Process 1 (sh) waited for 6 ticks, promoted to queue 0
```

- **Lottery result** — printed every time the lottery in **queue 0** picks a winner:

```
LOTTERY: Process 1 (sh) won in queue 0 with tickets 13
```

Bonus Task

Bonus: Multiple CPUs

Extend your implementation so the same MLFQ scheduler works correctly when `CPUS` is greater than 1, rather than assuming a single core.

Submission Guidelines

For this lab assignment, you must start with a fresh copy of the xv6 repository available at <https://github.com/shuaibw/xv6-riscv>. Clone the repository using the following command:

```
git clone https://github.com/shuaibw/xv6-riscv --depth=1
```

Using a fresh copy of this repository is important because, during the lab evaluation, you may be asked to modify and regenerate your patch file.

After cloning the repository, make all the necessary modifications and create any additional files required for the assignment. Do not commit your changes. Once you have completed and tested your implementation, generate a patch file containing only your changes using

the following commands from inside the xv6 repository:

```
git add --all
git diff HEAD > studentID.patch
```

Replace `studentID` with your own seven-digit student ID. For example, if your student ID is 2205192, the patch file should be named `2205192.patch`.

Submit only the patch file. Do not submit the entire xv6 repository, and do not compress or zip the patch file.

During the lab evaluation, **we will clone a fresh copy of the same xv6 repository** and apply your submitted patch using the following command:

```
git apply studentID.patch
```

Before submitting, verify that your patch works correctly by applying it to another fresh copy of the repository using the same procedure that will be followed during the lab evaluation.

Please DO NOT COPY solutions from anywhere (your friends, seniors, internet, etc.). Any form of plagiarism (irrespective of source or destination), will result in getting -100% marks in this assignment. You have to protect your code.

Mark Distribution

Task	Sub-task	Marks
MLFQ Scheduler Core	Queue transitions (top-down search, demotion on time-limit exhaustion, promotion on voluntary yield)	15
	Lottery scheduling in queue 0 (correct probability, non-floating-point procedure, ticket decrement and reset)	20
	Round-robin scheduling in queue 1	5
	Aging mechanism (promotion after <code>WAIT_THRESH</code> ticks)	15
System Calls	<code>settickets</code>	5
	<code>getpinfo</code> and <code>struct pstat</code>	10
	Ticket inheritance on <code>fork()</code>	5
Testing & Logging	<code>dummyproc</code>	5
	<code>testprocinfo</code>	5
	Logging messages (<code>DEMO</code> , <code>PROMO</code> , <code>BOOST</code> , <code>LOTTERY</code>)	10
Proper submission		5
Total		100
Bonus: Multiple CPUs		10